HW 2 Core: Projectile Clears a Wall

Physics Programming, Week 2

Submission: submit the fragment in the Week 2 core CodeRunner question on the course Moodle page.

Core homework: does a projectile clear a wall?

We launch from ground level with speed v_0, angle θ, and gravity magnitude $g > 0$, with $+x$ toward the wall and $+y$ upward. In the uniform-gravity, no-drag model,

$$v_x = v_0 \cos\theta, \quad t_w = x_w / v_x, \quad y_w = v_0 \sin\theta \, t_w - \tfrac{1}{2} g t_w^2, \quad v_{y,w} = v_0 \sin\theta - g t_w.$$

We require $v_x > 0$ before dividing. We compare y_w with the wall height, and use $v_{y,w}$ to classify rising or falling. Equality in height counts as clearing; zero vertical velocity belongs to the published falling category. The computed free-flight height may be negative: for a non-negative wall height this means failure to clear, not motion physically continuing through the ground.

Physics box: projectile at a wall

A projectile launched from the origin is pulled downward by gravity, so its horizontal speed stays constant while its vertical speed decreases. In uniform gravity with no drag, $v_x = v_0 \cos\theta$ and $v_y(t) = v_0 \sin\theta - gt$, and the height at horizontal distance x_w is $y_w = v_0 \sin\theta \, t_w - \frac{1}{2} g t_w^2$ with $t_w = x_w / v_x$. Here v_0 is the launch speed in m/s, θ the angle above the horizontal in degrees, $g > 0$ the gravity magnitude in m/s^2, x_w and y_w lengths in metres, t_w a time in seconds, and $v_{y,w}$ a vertical velocity in m/s. We take $+x$ toward the wall and $+y$ upward, launch from ground level, and require $v_x > 0$. The model assumes a point mass, flat ground, uniform gravity, and no air resistance. For $v_0 = 20$ m/s, $\theta = 45°$, and $x_w = 10$ m it gives $y_w = 7.5475$ m. A negative computed height means the free-flight parabola crossed the ground before the wall; it is not motion continuing underground. Air drag, spin, wind, and any collision with the wall lie outside this model.

We first write a prediction for one ordinary case and one boundary. Next we implement the scalar fragment, validate its documented domain, and compare the returned labels and Boolean exactly. We check heights approximately using the published tolerance. Finally we explain how the clearance decision and vertical-state decision differ: a projectile can clear while falling. The assignment sheet is the place to look for the precise input names, the tests, the convention for the returned wrapper values, and the submission instructions; neither student-defined functions nor loops are needed.

Task

A projectile starts at the origin with speed `speed_m_s` and launch angle `angle_deg`. Using the analytic, no-drag model, calculate its height at a vertical wall and decide whether it is rising or falling there and whether it clears the wall. Use $+x$ horizontally and $+y$ upward, $g > 0$, and angles converted from degrees to radians. At the wall, $t = x/(v\cos\theta)$, $y = v\sin\theta\, t - gt^2/2$, and $v_y = v\sin\theta - gt$. Equality with the wall counts as clearing; $v_y = 0$ is `"falling"` by contract. We assume uniform gravity and no drag; this does not model impact with the wall or ground.

Expected submission

Submit a fragment only (functions are taught in Week 5). The supplied wrapper returns three values with these types and possible values:

- `height_at_wall` (a finite `Real`, normally `Float64`): the calculated height in metres. It may be negative when the projectile lands before the wall.
- `vertical_state` (`String`): exactly `"rising"` when the vertical velocity is positive, or `"falling"` otherwise. Zero vertical velocity is `"falling"`.
- `clears` (`Bool`): `true` when `height_at_wall >= wall_height_m`, and `false` otherwise. Equality at the wall counts as `true`.

Validate finite non-Boolean real inputs: the launch angle is in $[0°, 90°)$, speed, wall distance, and wall height are nonnegative, gravity is positive, and horizontal velocity must be positive. (In Julia, `Bool` is a `Real`, so reject it explicitly.) Invalid inputs must `throw(ArgumentError(...))`, not print a warning. Assign all three outputs; print nothing.

```
# supplied: speed_m_s, angle_deg, wall_x_m, wall_height_m, gravity_m_s2
# assign: height_at_wall, vertical_state, clears
```

How to enter your answer

The CodeRunner answer box already contains the supplied input names and the three output names. Leave that structure unchanged. Replace only the line or comment that asks you to write the fragment. Do not define `wall_clearance` yourself, do not add a `return` statement, and do not print anything. CodeRunner calls the supplied wrapper and reads the three variables you assign.

Suggested calculation order

For your first CodeRunner exercise, follow this order:

1. **Check each input.** Use an `if`/`elseif` chain or separate `if` statements. Each value must be a finite `Real` that is not a `Bool`. The allowed ranges are

$$\texttt{speed_m_s} \geq 0, \quad 0 \leq \texttt{angle_deg} < 90,$$
$$\texttt{wall_x_m} \geq 0, \quad \texttt{wall_height_m} \geq 0, \quad \texttt{gravity_m_s2} > 0.$$

   If a check fails, execute `throw(ArgumentError("..."))`. The message is yours; the exception type matters.

2. **Convert the angle before using `sin` or `cos`.** Either use `sind` and `cosd`, or compute `theta = deg2rad(angle_deg)`.

3. **Compute the horizontal velocity and check it before dividing.** Set `vx = speed_m_s * cos(theta)`. If `vx` is not finite and positive, throw `ArgumentError`; otherwise `vx > 0` is guaranteed for the division. Then compute

$$t = \frac{\texttt{wall_x_m}}{\texttt{vx}}, \qquad \texttt{height_at_wall} = \texttt{speed_m_s}\sin\theta\, t - \tfrac{1}{2}\texttt{gravity_m_s2}\, t^2.$$

4. **Compute the vertical velocity and classify it.** Set `vy = speed_m_s * sin(theta) - gravity_m_s2 * t`. Assign `vertical_state = "rising"` when `vy > 0`; otherwise assign `"falling"`, including `vy == 0`.

5. **Set the Boolean.** Assign `clears = height_at_wall >= wall_height_m`. Use `>=`, not `>`, because an exact hit on the top edge counts as clearing.

6. **Stop.** Do not print, return a tuple, or change the supplied variables. The platform reads `height_at_wall`, `vertical_state`, and `clears` after your fragment runs.

Before you submit: check these cases

All cases use `gravity_m_s2 = 9.81`. These are useful hand checks; the first three are close to the public tests. The height is the returned number, not a printed line.

speed	angle	wall x	wall h	`height_at_wall`	`vertical_state`	`clears`
20	45	10	7	7.5475	`"rising"`	`true`
20	45	30	8	7.9275	`"falling"`	`false`
20	0	0	0	0.0	`"falling"`	`true`
20	45	50	2	-11.3125	`"falling"`	`false`

The second case shows that a projectile can be falling and still hit the wall. The third is the exact boundary: the calculated height equals the wall height, so `clears` is `true` and the vertical state is `"falling"` because `vy == 0`. The fourth has a negative calculated height; that is allowed by the contract and means the projectile would have landed before the wall.

Also test the rejected values. At least one of these must throw `ArgumentError`:

- negative speed, or zero speed when the horizontal velocity would not be positive;
- a negative angle, or an angle of 90° or more;
- negative wall distance or negative wall height;
- zero or negative gravity;
- `NaN` or `Inf` in any input;
- a Boolean value such as `true` in place of a number.

Common beginner mistakes

- Using `sin(angle_deg)` or `cos(angle_deg)` while the value is in degrees. Use `sind`/`cosd`, or convert with `deg2rad`.
- Dividing by `vx` before checking that it is positive.
- Checking `value isa Real` but forgetting that `Bool` is also a `Real`; reject `Bool` explicitly.
- Writing `clears = height_at_wall > wall_height_m`, which loses the exact top-edge case.
- Rounding the height before assigning or comparing it. Return the full calculated value; the tests allow ordinary floating-point differences.
- Printing a result or adding `return height_at_wall, vertical_state, clears`. The wrapper owns the return, and output is not part of this contract.
- Using the wrong string case, such as `"Rising"`, or assigning the string `"true"` instead of the Boolean `true`.

When a CodeRunner check fails, read the failing case and the reported expected and actual values. Compare the input order, the angle unit, the sign of gravity, the `vx` check, and the boundary convention. Fix one issue and run Check again; the staff tests use the same contract but different ordinary and boundary values.

How is this exercise checked?

The public tests call the wrapper with an ordinary rising case, a falling case, an exact top-edge case, and representative invalid inputs. Heights are compared with a tolerance of `1e-12` m; the state string and Boolean are compared exactly. The staff tests use the same published contract and add other ordinary, boundary, and invalid values. They do not introduce a new input name, a new output, or a different return convention. The complete public test body is in the optional Advanced section.

Advanced (optional)

The wrapper and the `@test` checks belong to the platform, and we do not write them this week. A wrapper is a supplied function that calls the submitted code and returns its results; a test written with `@test` states a claim that has to hold, while `@test_throws` states that an invalid input must be rejected. Reading them shows us how our work is checked the same way every time. The complete public test body is:

```
@testset "week02 core public" begin
    h, state, clears = wall_clearance(20.0, 45.0, 10.0, 7.0)
    @test isapprox(h, 7.5475; atol=1e-12); @test state == "rising";
   @test clears
    h, state, clears = wall_clearance(20.0, 45.0, 30.0, 8.0)
    @test state == "falling"; @test !clears
    @test wall_clearance(10.0, 0.0, 0.0, 0.0) == (0.0, "falling", true)
    @test_throws ArgumentError wall_clearance(-1.0, 30.0, 2.0, 1.0)
    @test_throws ArgumentError wall_clearance(10.0, 30.0, 2.0, -1.0)
    @test_throws ArgumentError wall_clearance(10.0, 90.0, 2.0, 1.0)
    @test_throws ArgumentError wall_clearance(true, 30.0, 2.0, 1.0)
end
```